

Machine Learning — Complete Mid-Semester Study Guide

BITS Pilani WILP · AIMLCZG565 Machine Learning

Compiled from CS1–CS6 companion docs, 6 years of MidSem papers & sample QPs

 Syllabus: CS1 → CS6 · Up to Decision Trees

START HERE — Read This First (Step-by-Step)

New to this guide? Follow these steps before diving into the numbered sections.

Step 1: How to use this file

1. Open **ML_Final_Study_Guide.html** in any browser (Chrome/Safari/Firefox).
2. Use the **left navigation panel** to jump to any topic. It tracks your scroll position.
3. All formulas render with MathJax — no extra app needed.
4. **Study method per topic:** Read concept → cover the answer → try the example on paper → reveal → move on.

Step 2: What does the MidSem look like?

Every MidSem paper has **5–6 questions, 25–30 marks total, 2 hours, Closed Book:**

Question	Topic	Typical Marks	Where to study
Q1	Introduction to ML (Mitchell's definition, types of learning)	4–6	Part I
Q2	ML Workflow & Data Quality (find 5–6 issues in a dataset)	3–6	Part II
Q3	Linear Regression / Regularization / Bias–Variance	4–6	Part III
Q4	Logistic Regression / Confusion Matrix / Feature Scaling	4–6	Part IV
Q5	Evaluation Metrics (Precision, Recall, F1)	4–6	Part V

Question	Topic	Typical Marks	Where to study
Q6	Decision Trees (Entropy, Information Gain, root node)	4–6	Part VI

Step 3: If you only have 2 hours to study

⚡ CRASH-COURSE ORDER

1. **START HERE** — understand the exam pattern (10 min)
2. **Mitchell's T/P/E** — appears in every single paper (10 min)
3. **Data Quality Issues** — memorise the 6 types (15 min)
4. **Bias–Variance & Regularization** — very high–yield (20 min)
5. **Confusion Matrix calculations** — guaranteed question (20 min)
6. **Entropy & Information Gain** — practice one full example (25 min)
7. **Formula Sheet** — scan and memorise key formulas (20 min)

Part I — Introduction to Machine Learning

Sections 1–2 · Always Q1 on the exam

📖 Starting Guide — Part I

What this part covers: What ML is, how it's defined, the 3 families of learning, and how to set up a learning problem.

Guaranteed exam question: "Tom Mitchell defined ML as... In this setting, what is E, T, and P?" — this appears in almost every paper.

Before you start, know:

- ML = making a computer improve at a task by learning from examples — no explicit programming.
- The 3 families: **Supervised** (labelled data), **Unsupervised** (find hidden structure), **Reinforcement** (learn from rewards).
- Within supervised: **Classification** (predict a category) vs **Regression** (predict a number).

Try this first: "A spam filter learns which emails are spam from 10,000 labelled examples." → Supervised learning, Classification. T = classify email as spam/not. P = accuracy. E = labelled email dataset.

1. What is Machine Learning?

MITCHELL'S DEFINITION (THE MOST IMPORTANT THING TO MEMORISE)

"A computer program is said to learn from **experience E** with respect to some **task T** and some **performance measure P**, if its performance on T, as measured by P, improves with experience E."

- **T (Task)** — what the system does (e.g., classify emails, predict house price)
- **P (Performance)** — how you measure success (accuracy, MSE, F1-score)
- **E (Experience)** — the data the system learns from (training dataset)

WORKED EXAMPLE — IDENTIFY T, P, E

Scenario: A self-driving car learns to drive safely by observing millions of hours of human driving and receives a reward signal when it reaches the destination without incident.

- | | |
|---|---|
| T | Task: Drive safely in urban environments (navigate roads, avoid obstacles, obey traffic rules) |
| P | Performance: Number of successful trips without incidents; distance driven safely; time to destination |
| E | Experience: Millions of hours of human driving data + reward signal from real-world driving outcomes |
|  | Type of learning: Reinforcement Learning (gets a reward/feedback, not pre-labelled data) |

Why ML instead of classical programming?

Classical Programming	Machine Learning
You write explicit rules	System learns rules from data

Classical Programming	Machine Learning
Rules break when world changes	Model adapts with new data
Good when rules are known	Good when rules are hard to articulate (images, speech)

2. Types of Learning

💡 THE THREE FAMILIES

- **Supervised Learning** — data has labels (y). Goal: learn $f(x) \rightarrow y$.
 - *Classification*: y is a category (spam/not spam, disease/healthy)
 - *Regression*: y is a number (house price, temperature)
- **Unsupervised Learning** — no labels. Goal: find structure in x .
 - *Clustering*: group similar items (K-means)
 - *Density estimation*: model distribution of data
- **Reinforcement Learning** — agent takes actions, receives rewards. Goal: maximise cumulative reward.

Scenario	Type	Sub-type
Predict insurance claim amount from age, vehicle type	Supervised	Regression
Classify sentiment of reviews as positive/negative/neutral	Supervised	Classification
Group DNA sequences into clusters to find disease links	Unsupervised	Clustering
Self-driving car improves over millions of miles driven	Reinforcement	—
Detect job applicants who lie using density estimation	Supervised (generative)	Classification
Predict tomorrow's rainfall in mm	Supervised	Regression

💡 DISCRIMINATIVE VS GENERATIVE CLASSIFIERS

Discriminative (e.g., Logistic Regression): directly models $P(y|x)$ — "given these features, what class?"

Generative (e.g., Naïve Bayes): models $P(x|y)$ — "how was this data generated for each class?"
 Needed when you want density estimation or to detect outliers ($P(x|y)$ required).

Exam tip: If asked "classify AND detect outliers via density estimation" → recommend **Generative** classifier.

[↑ Back to top](#)

Part II — ML Workflow & Data

Sections 3–7 · Always appears as Q2 — find issues in a messy dataset; data splitting tested in 2024

Starting Guide — Part II

The most reliable exam question: You are given a small, deliberately messy dataset table.
"Identify at least 5 data quality issues and suggest how to resolve them."

Memorise these 6 issue types — find one of each in any table:

- 1. **Missing values** — a cell is blank
- 2. **Outliers** — a value is implausibly extreme (Age = 350, CGPA = 87.5)
- 3. **Inconsistent format** — same data written differently (dates: "15-Aug-2022" vs "2022-08-15")
- 4. **Duplicate rows** — same record appears twice
- 5. **Wrong data type / unit mismatch** — weight in "120lbs" vs rest in "kg"; email "dev[at]mail.com"
- 6. **Inconsistent encoding** — result stored as "Positive/Negative" in some rows but "1/0" in others

3. Data Quality Issues

CLASSIC EXAM DATASET — SPOT ALL ISSUES

This exact dataset (or a variant) has appeared in every single midsem paper from 2023–2026:

TXN-ID	NAME	AGE	WEIGHT	BLOOD GROUP	COVID RESULT
T003	Akbar	38	60kg	lam+ve	Positive

TXN-ID	NAME	AGE	WEIGHT	BLOOD GROUP	COVID RESULT
T005	THenali	22	78kg	B-ve	1
T007	Rajuu	350	48kg	O+ve	Positive
T008	HARI	32	120lbs	AB-ve	Negative
T009	Inba	25	(missing)	O+ve	0
T010	SysUsr789	20	68kg	O-ve	Negative

- 1 Outlier (Age=350, T007):** Age of 350 is biologically impossible. Resolution: Replace with mean age of other records, or flag for manual verification.
- 2 Unit mismatch (Weight=120lbs, T008):** All other weights are in kg; this one is in lbs. Resolution: Convert using 1 lb = 0.453 kg → 120 lbs ≈ 54.4 kg via data transformation.
- 3 Missing value (Weight missing, T009):** Weight cell is empty. Resolution: Impute with mean/median weight, or use KNN-imputation.
- 4 Inconsistent format/noise in Name (SysUsr789, T010):** Name field has an alphanumeric system ID instead of a person's name. Resolution: Flag for manual review and replace with correct name.
- 5 Inconsistent Blood Group encoding (lam+ve, T003):** "lam+ve" is not a valid blood group format; should be "AB+ve" or similar. Resolution: Apply binning/standardisation; replace with NULL or correct value.
- 6 Inconsistent COVID Result encoding (T005=1, T009=0):** Most records use "Positive/Negative" but T005 and T009 use "1/0". Resolution: Data transformation — standardise all to "Positive/Negative" (or all to 0/1).

4. Missing Values & Outliers

Handling Missing Values

Method	When to use	Risk
Delete row	Very few missing rows (<5%)	Loses data

Method	When to use	Risk
Mean/Median imputation	Numerical features, data is MAR	Reduces variance
Mode imputation	Categorical features	Introduces bias
KNN imputation	When relationships between features exist	Computationally expensive
Model-based imputation	Complex structured data	More assumptions

Detecting and Handling Outliers

💡 TWO MAIN DETECTION METHODS

Z-score: $z = \frac{x - \mu}{\sigma}$. Values with $|z| > 2$ or 3 are outliers. Assumes approximately normal distribution.

IQR method: $Q1 - 1.5 \times IQR$ to $Q3 + 1.5 \times IQR$. Any value outside this range is flagged. More robust than z-score.

Where $IQR = Q3 - Q1$ (interquartile range).

5. Feature Scaling

💡 WHY SCALE? (VERY FREQUENTLY TESTED)

Gradient descent uses step sizes proportional to the gradient. If one feature has range $[0, 1]$ and another $[0, 1,000,000]$, the gradient will be dominated by the large-scale feature. The contour lines become elongated ellipses, forcing many more iterations to converge.

Scaling makes gradient descent faster (fewer iterations to converge) by making cost-function contours more circular.

Statement	True or False?	Reason
Feature scaling guarantees convergence to global minimum	False	Local minima can still exist (e.g., neural nets). Scaling only speeds convergence, doesn't change topology.

Statement	True or False?	Reason
Feature scaling prevents gradient descent from getting stuck in local optima	False	Linear regression cost function is convex (no local optima). Scaling doesn't affect this.
Feature scaling speeds up gradient descent	True	More circular contours → fewer iterations needed.
Feature scaling prevents $X^T X$ from being singular	False	Singularity occurs from linearly dependent features, not from scale differences.

Two main scaling methods

MIN-MAX NORMALISATION

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Maps all values to $[0, 1]$. Sensitive to outliers.

Z-SCORE STANDARDISATION

$$x' = \frac{x - \mu}{\sigma}$$

Maps to mean=0, std=1. More robust when outliers exist.

⚠ MUST SCALE BEFORE K-NN AND GRADIENT DESCENT

k-NN uses distance — unscaled features with large ranges dominate the distance calculation. For instance, income (thousands) + age (tens) → income controls everything. **Always scale before k-NN.**

6. Feature Engineering

- **One-hot encoding** — convert categorical variables into binary columns (e.g., Blood Group → O+ve: 1/0, A+ve: 1/0, ...)
- **Binning** — group continuous values into ranges (Age → Young/Middle/Senior)
- **Polynomial features** — create x^2 , $x \cdot y$ features to capture non-linear relationships
- **Interaction terms** — multiply two features to capture joint effects
- **Removing redundant features** — if Age and Date-of-Birth both present, keep one (Age = $f(\text{DOB})$)

7. Data Splitting & Validation

💡 THE CORE PRINCIPLE — NEVER TOUCH THE TEST SET

A model is evaluated honestly only on data it has *never seen during training*. The standard workflow:

- 1. **Train set (≈60–70%)** — the model learns from this data.
- 2. **Validation set (≈15–20%)** — used to tune hyperparameters and choose models (optional but best practice).
- 3. **Test set (≈15–20%)** — locked away; evaluated only once at the very end to report final performance.

Why keep test set separate? If you peek at the test set to make modelling decisions, test accuracy no longer reflects real-world performance — you've *leaked* information from the test set into training decisions.

Split Strategy	When to use	Key point
Random split	Large dataset, balanced classes	Simple; use <code>random_state</code> for reproducibility
Stratified split	Imbalanced classes	Each split preserves the original class proportions. Essential when one class is rare (e.g., 5% positive).
K-fold cross-validation	Small datasets	Data split into k folds; model trained k times (each fold is the val set once). Average score is more reliable than one split.
Stratified K-fold	Small + imbalanced datasets	Combines stratification with k-fold; every fold has the same class ratio.

🎯 STRATIFIED SPLIT — EXAM FAVOURITE (2024 PAPERS)

Q: You have a loan-approval dataset: 87% rejected, 13% approved. You randomly split 70/30. What risk does this create and what should you do instead?

Risk: A random split may put nearly all approved cases in training or test — the splits no longer represent the true class distribution. The model could be trained with very few approved examples.

Fix: Stratified split. Ensures both 70% train and 30% test contain approximately 87% rejected / 13% approved each, preserving the real-world distribution.

CROSS-VALIDATION — WHY IT MATTERS

Q: You train a model on a 1000-sample dataset and get 92% accuracy on the test set. Your colleague used 5-fold CV and got $84\% \pm 3\%$. Which estimate is more trustworthy?

- 1 **Single 80/20 split:** The 92% is based on 200 test samples — it can be lucky or unlucky depending on which 200 examples ended up in the test set. High variance estimate.
 - 2 **5-fold CV:** Each of the 5 test folds uses 200 different samples. The model trains and evaluates on all 1000 data points across 5 runs. The $\pm 3\%$ gives a confidence interval — much more representative.
- ✓ **5-fold CV is more trustworthy.** The 92% single-split estimate is likely inflated (overly optimistic). CV's lower but stable estimate better reflects real generalisation performance.

[↑ Back to top](#)

Part III — Linear Regression

Sections 8–14 · Appears in Q3; bias–variance, regularization, and polynomial regression always tested

Starting Guide — Part III

In one sentence: Fit a line (or hyperplane) to data by minimising the average squared prediction error, then dial in regularization to prevent overfitting.

Key questions the exam asks:

- Given training/validation error patterns → does the model underfit or overfit?
- Large dataset → gradient descent or normal equation? (Answer: gradient descent — $O(n^3)$ matrix inversion is too slow)
- Ridge vs Lasso — which reduces coefficients to zero? (Lasso)
- What does increasing λ do to bias and variance?

8. Linear Regression

💡 THE MODEL

Predict a continuous value $\hat{y}$ from features $\mathbf{x}$:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n = \boldsymbol{\theta}^T \mathbf{x}$$

θ_0 = intercept (bias term), $\theta_1, \dots, \theta_n$ = weights (slopes).

Interpretation of a coefficient: If $\theta_j = 2.5$, then for every unit increase in x_j , the predicted output increases by 2.5 (holding all other features constant).

9. Cost Function & Gradient Descent

Mean Squared Error (MSE) — the cost function

$$J(\boldsymbol{\theta}) = \frac{1}{2m} \sum_{i=1}^m \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

where m = number of training examples. The factor $\frac{1}{2}$ is a convenience that cancels during differentiation.

💡 THREE FLAVOURS OF GRADIENT DESCENT

Variant	Update uses	Pros	Cons
Batch GD	All m examples per step	Smooth convergence; guaranteed for convex	Slow on large datasets
Stochastic GD (SGD)	1 random example per step	Fast; can escape local minima	Noisy; may not converge exactly
Mini-batch GD	k examples per step ($k=32-256$)	Balance of speed & stability; GPU-efficient	Extra hyperparameter (batch size)

Gradient Descent Update Rule

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial J}{\partial \theta_j}$$

For linear regression (MSE cost): $\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) \cdot x_j^{(i)}$

🔗 LEARNING RATE EXAM TRAP

If $J(\theta)$ increases over iterations $\rightarrow$ learning rate α is *too large* $\rightarrow$ **decrease α** . The algorithm is overshooting the minimum.

If $J(\theta)$ decreases very slowly $\rightarrow \alpha$ may be too small $\rightarrow$ try a larger α (but not too large).

Rule of thumb: Try $\alpha = 0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1$ and plot J vs iterations.

10. Normal Equation

💡 CLOSED-FORM SOLUTION

$$\theta = (X^T X)^{-1} X^T y$$

No iterations, no learning rate — computes optimal θ directly.

Dimensions check (common exam question): If $m=28$ training examples, $n=4$ features (plus bias):

- X is **28×5** (m rows $\times$ ($n+1$) cols including $x_0=1$)
- y is **28×1**
- θ is **5×1**

🔗 WHEN TO USE NORMAL EQUATION VS GRADIENT DESCENT?

Normal Equation: Computing $(X^T X)^{-1}$ costs $O(n^3)$ where n = number of features. For large n (e.g., $n = 300,000$), this is computationally infeasible.

Gradient Descent is preferred when: n is very large (e.g., $>10,000$ features), or m is very large (millions of examples).

Normal Equation preferred when: n is small ($\leq \sim 10,000$) — exact answer, no learning rate to tune.

📌 EXAM QUESTION — GD VS NORMAL EQUATION

Q: You have $n=2,000,000$ instances and $m=300,000$ features. Which approach for linear regression — gradient descent or normal equation? Why?

- ✅ **Gradient Descent.** Computing $(X^T X)^{-1}$ for 300,000 features requires inverting a $300,000 \times 300,000$ matrix. Matrix inversion is $O(n^3)$, which is approximately $(3 \times 10^5)^3 = 2.7 \times 10^{16}$ operations — completely infeasible. Gradient descent scales much better and only requires computing gradients incrementally.

11. Polynomial Regression

💡 MOTIVATION — FITTING NON-LINEAR DATA

Real data is rarely perfectly linear. **Polynomial regression** extends linear regression by adding powers of input features, enabling the model to fit curved relationships while still using the linear regression framework.

For a single feature x , a degree- d polynomial model adds features $x, x^2, x^3, \dots, x^d$:

$$\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2 + \dots + \theta_d x^d$$

This is still a *linear model* in the **parameters θ** — we just engineered new features. We can use all the same tools: MSE cost, gradient descent, normal equation.

Degree	Model complexity	Risk	Typical behaviour
d = 1	Linear (straight line)	Underfitting (high bias)	Poor fit if data is curved
d = 2–3	Moderate	Balanced	Usually captures real patterns well
d = 8+	Very complex	Overfitting (high variance)	Fits training data perfectly; oscillates wildly on new data

🌟 POLYNOMIAL DEGREE & REGULARIZATION — 2024 EXAM STYLE

A typical 2024 MidSem question compared four models (A: linear, B: degree-8, C: degree-8 + Ridge, D: degree-8 + Lasso) and asked to rank their training RMSE, test RMSE, and identify high bias vs high variance:

- **Model A (linear on curved data):** High training RMSE AND high test RMSE → **high bias (underfitting)**.
- **Model B (degree-8, no regularization):** Very low training RMSE, high test RMSE → **high variance (overfitting)**.
- **Model C (degree-8 + Ridge):** Slightly higher training RMSE than B; lower test RMSE — penalises all large weights evenly. **Good generalisation if λ well-tuned.**
- **Model D (degree-8 + Lasso):** Sets many of the degree-8 coefficients exactly to zero → effectively reduces the polynomial degree. **Automatic model simplification + feature selection.**

RMSE ranking (training, lowest to highest): $B < C \approx D < A$.

RMSE ranking (test, lowest to highest): $C \approx D < A < B$.



CHOOSING THE RIGHT POLYNOMIAL DEGREE

Q: You fit polynomial models of degree 1 through 10 to a dataset. Training MSE keeps decreasing. Validation MSE decreases until degree 4 then increases. What should you do?

- 1 **Degree 4 is the sweet spot.** Below degree 4: validation MSE is still high → model is too simple (underfitting). Above degree 4: validation MSE rises even though training MSE drops → model is memorising noise (overfitting).
 - 2 **Training MSE always decreases** as degree increases — a model with more parameters can always fit training data better. This is why you must never use training error alone to choose model complexity.
- ✓ **Choose degree 4.** To confirm: apply regularization (Ridge/Lasso) and compare test RMSE across degrees. Alternatively, use cross-validation to estimate generalisation error at each degree.

12. Bias–Variance Trade-off

💡 THE CORE IDEA

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

- **Bias** = error from wrong assumptions (model too simple). High bias → underfitting → high training AND test error.

- **Variance** = error from sensitivity to training data fluctuations (model too complex). High variance → overfitting → low training error but high test error.
- **Irreducible noise** = inherent randomness in data — cannot be reduced.

Observation	Diagnosis	Fix
Training error $\approx$ validation error, both HIGH	High bias (underfitting)	More features, more complex model, reduce λ
Training error LOW, validation error HIGH	High variance (overfitting)	More training data, regularization (increase λ), simpler model, early stopping
Training error $\approx$ validation error, both LOW	Well-fitted model ✓	Deploy!

CLASSIC EXAM SCENARIO — RIDGE REGRESSION

Q: You use Ridge Regression. Training error and validation error are almost equal and fairly high. Does the model have high bias or high variance? Should you increase or decrease λ ?

- 1 **Diagnosis:** Both errors are high and equal → the model is **underfitting** → **high bias**.
 - 2 **What to do: Decrease λ .** Higher λ increases regularization → more bias (simpler model). We need less regularization to allow the model to fit the data better.
- ✓ **Rule:** $\uparrow \lambda \rightarrow \uparrow$ bias, $\downarrow$ variance. $\downarrow \lambda \rightarrow \downarrow$ bias, $\uparrow$ variance. Cross-validation finds the sweet spot.

13. Regularization (Ridge & Lasso)

WHY REGULARIZATION?

Overfitting happens when a model learns noise in training data. Regularization adds a **penalty term** to the cost function that discourages large coefficients (large weights = overfit signal).

L2 REGULARIZATION — RIDGE

$$J(\theta) = \text{MSE} + \lambda \sum_{j=1}^n \theta_j^2$$

L1 REGULARIZATION — LASSO

$$J(\theta) = \text{MSE} + \lambda \sum_{j=1}^n |\theta_j|$$

Penalises the *squared* size of coefficients.
 Shrinks all coefficients towards zero but **never exactly to zero**. Uses all features.
 Use when: all features are potentially relevant.

Penalises the *absolute* size of coefficients. Can set some coefficients **exactly to zero** → **automatic feature selection**. Produces sparse models.
 Use when: many features, expect some to be irrelevant.

	Ridge (L2)	Lasso (L1)
Penalty	$\lambda \ \boldsymbol{\theta}\ _2^2$	$\lambda \ \boldsymbol{\theta}\ _1$
Coefficients set to zero?	No (approaches zero)	Yes (exact zeros possible)
Feature selection?	No	Yes
Best for	All features matter (multicollinearity)	High-dim, sparse, irrelevant features present

🎯 WHICH REGULARIZATION? — EXAM FAVOURITE

Q: "Dataset with hundreds of predictors, many redundant or irrelevant" → which regularization?

→ **Lasso (L1)**. It drives irrelevant feature coefficients exactly to zero, effectively performing automatic feature selection and producing an interpretable sparse model.

14. R^2 & Model Evaluation for Regression

💡 R-SQUARED (COEFFICIENT OF DETERMINATION)

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

- $R^2 = 1.0$ → perfect fit (model explains all variance)
- $R^2 = 0.0$ → model performs no better than predicting the mean
- $R^2 < 0$ → model is worse than predicting the mean (possible with regularization)

Interpretation: $R^2 = 0.76$ means the model explains 76% of the variance in y . The remaining 24% is unexplained by the model.

Part IV — Classification & Logistic Regression

Sections 15–20 · Appears as Q4; sigmoid behaviour, weight interpretation, and the LR→classification gap frequently tested

Starting Guide — Part IV

In one sentence: Logistic regression uses a sigmoid to squash a linear score into a probability, then optimises log-loss via gradient descent.

Key questions the exam asks:

- Why can't you use linear regression directly for classification? (unbounded outputs, non-convex cost)
- $J(\theta)=0 \rightarrow$ is the model good? (No — it's overfitting the training data)
- Large weights $\rightarrow$ what happens to the sigmoid? (It becomes a step function $\rightarrow$ overfitting)
- Why must you scale features before logistic regression?
- Interpret a weight: " $\theta_1 = 0.5$ " $\rightarrow$ what does it mean in terms of log-odds?

15. Why Linear Regression Fails for Classification

THE PROBLEM — OUTPUTS ARE UNBOUNDED

Linear regression outputs $\hat{y} = \theta^T \mathbf{x}$, which can take any value from $-\infty$ to $+\infty$. For classification, we need a **probability**, which must stay in $[0, 1]$.

Two concrete problems when you force linear regression onto binary labels:

- **Probabilities go out of range:** Linear output can produce values like -0.3 or 1.7 — meaningless as probabilities.
- **Non-convex cost function:** Using MSE loss with a sigmoid output creates a loss surface with many local minima. Gradient descent can get stuck and fail to find the global minimum.
- **Sensitive to outliers:** An extreme training example can pull the linear regression line significantly, shifting the decision boundary in unpredictable ways.

💡 THE FIX — LOGISTIC REGRESSION

Instead of predicting y directly, we model the **probability** that $y = 1$:

$$P(y = 1|\mathbf{x}) = \sigma(\boldsymbol{\theta}^T \mathbf{x}) = \frac{1}{1 + e^{-\boldsymbol{\theta}^T \mathbf{x}}}$$

The sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$ **squashes any real number into (0, 1)**, making the output a valid probability.

We then use **cross-entropy (log-loss)** instead of MSE — this gives a convex loss surface, guaranteeing gradient descent finds the global minimum.

	Linear Regression on Binary Labels	Logistic Regression
Output range	$-\infty$ to $+\infty$ ❌	(0, 1) ✓
Interpretation	Not a probability ❌	$P(y=1 x)$ ✓
Cost function	MSE $\rightarrow$ non-convex ❌	Log-loss $\rightarrow$ convex ✓
Sensitive to outliers?	Yes (bad) ❌	Less sensitive ✓

🎯 CLASSIC EXAM Q: "WHY NOT USE LINEAR REGRESSION FOR CLASSIFICATION?"

Full marks answer must mention **three things**: (1) outputs not bounded to [0,1] — invalid probabilities; (2) MSE on binary labels creates non-convex cost with local minima; (3) the model is sensitive to outliers in a way that distorts the decision boundary.

16. Classification Basics

💡 DECISION BOUNDARY

A classification model learns a **decision boundary** — a surface that separates classes in feature space.

- **Linear boundary**: a straight line (2D), plane (3D), or hyperplane (nD). Linear models draw this.
- **Non-linear boundary**: curves, circles, complex shapes. Decision trees and neural networks can draw these.

Logistic regression draws a **linear decision boundary** in the original feature space.

17. Sigmoid Function & Logistic Regression

💡 THE SIGMOID (S-CURVE)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Maps any real number z to the range $(0, 1)$. Output is interpreted as a **probability**.

- $z \rightarrow +\infty$: $\sigma(z) \rightarrow 1$ (predict class 1)
- $z \rightarrow -\infty$: $\sigma(z) \rightarrow 0$ (predict class 0)
- $z = 0$: $\sigma(z) = 0.5$ (decision boundary)

For logistic regression: $z = \boldsymbol{\theta}^T \mathbf{x} = \theta_0 + \theta_1 x_1 + \dots$, so:

$$P(y = 1 | \mathbf{x}) = \sigma(\boldsymbol{\theta}^T \mathbf{x}) = \frac{1}{1 + e^{-\boldsymbol{\theta}^T \mathbf{x}}}$$

Predict class 1 if $P(y=1|x) \geq 0.5$, i.e., when $\boldsymbol{\theta}^T \mathbf{x} \geq 0$.

12 34 SIGMOID & OVERFITTING — CLASSIC EXAM QUESTION

Q: Plot $\sigma(wx)$ for $w \in \{1, 10, 100\}$. How does increasing w cause overfitting?

1 **w=1:** Smooth, gradual curve — gentle transition from 0 to 1 over a wide range of x .

2 **w=10:** Steeper curve — transitions from 0 to 1 much more rapidly near $x=0$.

3 **w=100:** Almost a step function at $x=0$ — essentially 0 for $x < 0$, 1 for $x > 0$.

✓ **Why overfitting:** Very large weights make the sigmoid hyper-confident. Even a tiny distance from the decision boundary results in a probability near 0 or 1. The model perfectly memorises the training data boundary but generalises poorly — a classic sign of overfitting. Fix: L2 regularization penalises large weights.

18. Log-Loss (Cross-Entropy) & Training

💡 WHY NOT USE MSE FOR CLASSIFICATION?

MSE + sigmoid gives a non-convex cost function with many local minima. **Log-loss gives a convex cost** → gradient descent guaranteed to find the global minimum.

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]$$

where $\hat{p}^{(i)} = \sigma(\theta^T \mathbf{x}^{(i)})$.

Intuition: When $y=1$, we want $\hat{p}$ close to 1 → $-\log(1) = 0$ (no penalty). If $\hat{p}$ is near 0 → $-\log(0) = \infty$ (huge penalty). When $y=0$, the second term does the same work.

J(θ)=0 — TRUE OR FALSE STATEMENTS (APPEARS IN EVERY PAPER)

Q: You train logistic regression and achieve $J(\theta_0, \theta_1) = 0$. Which statements are true?

- a) The model will work perfectly on unseen data.
- b) $h_{\theta}(\mathbf{x}^{(i)}) = y^{(i)}$ for every training example.
- c) θ_0 and θ_1 must both be 0.
- d) $J(\theta)$ cannot be 0.

- a **FALSE.** $J=0$ means perfect fit on training data → the model is **overfitted**. It memorised training data and will likely perform poorly on unseen data.
- b **TRUE.** If $J=0$, the log-loss sum equals zero, which means every term must be 0, which means $h_{\theta}(\mathbf{x}^{(i)}) = y^{(i)}$ exactly for all training examples.
- c **FALSE.** $\theta=0$ gives $h_{\theta}(x)=0.5$ for all x — not necessarily 0 loss. The parameters can take any values that perfectly separate the training classes.
- d **FALSE.** J can be 0 if the training data is linearly separable and large weights push all predictions to exactly 0 or 1.

19. Interpretation of Weights (Log-Odds)

COEFFICIENTS = CHANGE IN LOG-ODDS

Starting from the sigmoid and taking the log-odds (logit):

$$\text{logit}(P(y = 1)) = \log \frac{P(y = 1)}{P(y = 0)} = \theta^T \mathbf{x} = \theta_0 + \theta_1 x_1 + \dots$$

So θ_j = the change in log-odds of the positive class for a one-unit increase in x_j (all other features fixed).

- $\theta_j > 0 \rightarrow$ increasing x_j increases probability of $y=1$
- $\theta_j < 0 \rightarrow$ increasing x_j decreases probability of $y=1$
- $\theta_j \approx 0 \rightarrow$ feature x_j has little effect

🌟 FEATURE SCALING & LOGISTIC REGRESSION

Q: Why is feature scaling important for logistic regression?

Because logistic regression is trained via gradient descent. Without scaling, features with large ranges dominate the gradient and the algorithm takes much longer to converge (or may oscillate). Scaling ensures all features contribute roughly equally to each gradient step.

20. Multi-Class Classification

Strategy	How it works	When to use
One-vs-Rest (OvR)	Train k binary classifiers, one for each class vs all others. Predict the class whose classifier is most confident.	Default; works well in practice
Softmax (Multinomial)	Single model outputs a probability vector over all k classes. $\text{softmax}(z_k) = \frac{e^{z_k}}{\sum_j e^{z_j}}$ Probabilities sum to 1.	When classes are mutually exclusive

[↑ Back to top](#)

Part V — Evaluation Metrics

Sections 21–23 · Guaranteed Q5 — always involves confusion matrix calculations

📖 Starting Guide — Part V

The most calculable part of the exam. Every paper has a confusion matrix question — memorise the formulas and you will always get full marks.

The one thing to memorise: TP, FP, FN, TN are always relative to the class you call "Positive".

Quick drill: Given CM with TP=60, FP=120, FN=0, TN=20: Precision = $60/180 = 33.3\%$; Recall = $60/60 = 100\%$

21. The Confusion Matrix

💡 READING A CONFUSION MATRIX

	Predicted POSITIVE	Predicted NEGATIVE
Actual POSITIVE	TP (True Positive) ✓	FN (False Negative — missed)
Actual NEGATIVE	FP (False Positive — false alarm)	TN (True Negative) ✓

- **TP:** Correctly predicted positive — hit
- **FP:** Predicted positive but actually negative — false alarm / Type I error
- **FN:** Predicted negative but actually positive — missed / Type II error
- **TN:** Correctly predicted negative

22. Precision, Recall, F1-score, Specificity

ACCURACY

$$\frac{TP+TN}{TP+FP+FN+TN}$$

Misleading on imbalanced data!

PRECISION

$$\frac{TP}{TP+FP}$$

Of predicted positives, how many correct?

Minimise false alarms.

RECALL (SENSITIVITY / TPR)

$$\frac{TP}{TP+FN}$$

Of actual positives, how many caught? Minimise missed cases.

SPECIFICITY (TNR)

$$\frac{TN}{TN+FP}$$

Of actual negatives, how many correctly identified?

F1-SCORE

$$\frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

FALSE POSITIVE RATE (FPR)

$$\frac{FP}{FP+TN} = 1 - \text{Specificity}$$

Used to plot ROC curve.

Harmonic mean — balances both. Best for imbalanced data.

🎯 WHEN TO USE PRECISION VS RECALL?

- **Maximise Precision** when False Positives are costly — e.g., spam filter (don't flag important emails as spam), medical test where treatment has side effects.
- **Maximise Recall** when False Negatives are costly — e.g., cancer detection (don't miss any cancer), fraud detection (don't miss fraud).
- **Use F1** when you need balance between both.

📊 FULL CONFUSION MATRIX CALCULATION (CREDIT CARD FRAUD)

Given: Test set of 200 transactions. The model produces:

	Predicted Fraud	Predicted Not Fraud
Actual Fraud	60	0
Actual Not Fraud	120	20

Calculate Precision, Recall, and Accuracy for the **Fraud class**.

1 Label TP, FP, FN, TN (Fraud = Positive):

TP = 60 (predicted Fraud, actual Fraud)

FN = 0 (predicted Not Fraud, actual Fraud — missed none!)

FP = 120 (predicted Fraud, actual Not Fraud — false alarms)

TN = 20 (predicted Not Fraud, actual Not Fraud)

2 Precision (Fraud): $\frac{TP}{TP+FP} = \frac{60}{60+120} = \frac{60}{180} = 33.3\%$

Only 1 in 3 fraud alerts is real — too many false alarms.

3 Recall (Fraud): $\frac{TP}{TP+FN} = \frac{60}{60+0} = 100\%$

Caught every single fraud transaction — perfect recall!

4 Accuracy: $\frac{60+20}{200} = \frac{80}{200} = 40\%$

Very low accuracy despite catching all fraud — model is biased towards predicting Fraud.

- ✓ **Observations:** The classifier has perfect recall for fraud but terrible precision (120 false alarms for 60 real frauds). This would flood the investigation team. The trade-off between precision and recall must be tuned via the decision threshold.

Multi-class Precision & Recall

For a 3-class confusion matrix (Class A, B, C), to find precision/recall for Class B:

- **TP** = correctly predicted B (diagonal cell for B)
- **FP** = other classes predicted as B (other rows in B's predicted column)
- **FN** = B instances predicted as other classes (other columns in B's actual row)

⚠ ACCURACY PARADOX — CLASS IMBALANCE (EXAM FAVOURITE)

If 95% of loan applications are rejected, a model that *a/ways* predicts "reject" achieves **95% accuracy** — but is completely useless (it never detects approvals).

Accuracy is misleading on imbalanced datasets. Use Precision, Recall, F1, or AUC instead. This scenario appears in every paper that has a confusion matrix question.

Always compute F1 and check the confusion matrix when class distribution is skewed.

23. ROC Curve & AUC

💡 ROC CURVE

A ROC curve plots **True Positive Rate (Recall)** on the y-axis vs **False Positive Rate** on the x-axis as the classification threshold varies from 1 → 0.

- **AUC = 1.0:** Perfect classifier
- **AUC = 0.5:** Random classifier (diagonal line)
- **AUC > 0.7:** Acceptable; >0.8 good; >0.9 excellent

AUC = probability that a randomly chosen positive instance ranks higher than a randomly chosen negative instance.

📊 TPR/FPR AT DIFFERENT THRESHOLDS

Given: True labels = [1, 0, 1, 0, 1], predicted probabilities = [0.9, 0.7, 0.6, 0.3, 0.2]

P=3, N=2. Calculate TPR and FPR at threshold >0.8 and threshold >0.5.

- 1 **Threshold > 0.8:** Only prob=0.9 is predicted positive.

TP=1 (prob=0.9, true=1), FP=0, FN=2, TN=2

TPR = $1/3 \approx 0.33$; FPR = $0/2 = 0.0$

ROC point: (0.0, 0.33)

- 2 **Threshold > 0.5:** Probs 0.9, 0.7, 0.6 are predicted positive.

TP=2 (probs 0.9, 0.6 with true=1), FP=1 (prob=0.7 with true=0), FN=1, TN=1

TPR = $2/3 \approx 0.67$; FPR = $1/2 = 0.5$

ROC point: (0.5, 0.67)

- ✓ As threshold decreases: TPR increases (catch more positives) but FPR also increases (more false alarms). The ROC curve plots this trade-off.

[↑ Back to top](#)

Part VI — Decision Trees

Sections 24–27 · Always Q6; entropy + information gain calculation is guaranteed 4–6 marks

Starting Guide — Part VI

In one sentence: A decision tree repeatedly splits data by the attribute that gives the highest information gain (= greatest reduction in entropy).

The exam always asks you to: compute entropy of the target class, compute entropy of each feature, compute information gain, pick the root node (the one with highest IG).

Entropy formula: $H(S) = - \sum_c p_c \log_2 p_c$ — where p_c is the fraction of class c in set S .

Key facts to memorise:

- $H = 0$ (pure node, all same class), $H = 1$ (50-50 split on 2 classes, maximum impurity)
- $IG = H(\text{parent}) - \text{weighted average } H(\text{children})$
- Pick the attribute with the **highest IG** (= lowest weighted entropy) as root.

24. Decision Trees & ID3 Algorithm

💡 WHAT IS A DECISION TREE?

A tree-structured model where each **internal node** tests an attribute, each **branch** is an attribute value, and each **leaf node** is a class prediction.

ID3 algorithm: At each node, choose the attribute with the highest Information Gain → split the data → recurse. Stop when all instances in a node are the same class (pure leaf), or no attributes remain (use majority class).

25. Entropy & Information Gain — Full Calculation

💡 ENTROPY FORMULA

$$H(S) = - \sum_{c=1}^k p_c \log_2 p_c$$

where p_c = fraction of examples in class c , k = number of classes.

Key values:

- All one class: $H = 0$ (pure)
- 50-50 (2 classes): $H = 1.0$ bit (maximum impurity)
- 33-33-33 (3 classes): $H = \log_2(3) \approx 1.585$ bits

💡 INFORMATION GAIN FORMULA

$$IG(S, A) = H(S) - \sum_{v \in \text{values}(A)} \frac{|S_v|}{|S|} \cdot H(S_v)$$

where S_v = subset of S where attribute $A = v$. The second term is the **weighted average entropy** of the children after splitting on A .

📄 FULL ID3 EXAMPLE — ENTROPY & INFORMATION GAIN (DEC 2025 MIDSEM STYLE)

Given dataset (12 records): 8 "Major" crash severity, 4 "Minor". Consider attribute "Driver Condition" with values {Alcohol, Sober} and "Weather" with values {Good, Bad}.

Distribution: Alcohol: 4 Major, 1 Minor; Sober: 4 Major, 3 Minor; Good: 5 Major, 2 Minor; Bad: 3 Major, 2 Minor.

1 Parent Entropy $H(S)$:

$$p(\text{Major}) = 8/12, p(\text{Minor}) = 4/12$$

$$H(S) = -\frac{8}{12} \log_2 \frac{8}{12} - \frac{4}{12} \log_2 \frac{4}{12}$$

$$= -(0.667 \times (-0.585)) - (0.333 \times (-1.585)) = 0.390 + 0.528 = \mathbf{0.918 \text{ bits}}$$

2 Driver Condition — entropy of each branch:

$$\text{Alcohol (5 total: 4 Maj, 1 Min): } H = -\frac{4}{5} \log_2 \frac{4}{5} - \frac{1}{5} \log_2 \frac{1}{5} = 0.722$$

$$\text{Sober (7 total: 4 Maj, 3 Min): } H = -\frac{4}{7} \log_2 \frac{4}{7} - \frac{3}{7} \log_2 \frac{3}{7} = 0.985$$

$$\text{Weighted avg} = \frac{5}{12} (0.722) + \frac{7}{12} (0.985) = 0.301 + 0.574 = 0.875$$

$$\mathbf{IG(\text{Driver}) = 0.918 - 0.875 = 0.043}$$

3 Weather Condition — entropy of each branch:

$$\text{Good (7 total: 5 Maj, 2 Min): } H = -\frac{5}{7} \log_2 \frac{5}{7} - \frac{2}{7} \log_2 \frac{2}{7} = 0.863$$

$$\text{Bad (5 total: 3 Maj, 2 Min): } H = -\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} = 0.971$$

$$\text{Weighted avg} = \frac{7}{12} (0.863) + \frac{5}{12} (0.971) = 0.503 + 0.405 = 0.908$$

$$\mathbf{IG(\text{Weather}) = 0.918 - 0.908 = 0.010}$$

- ✓ **Choose root node:** $IG(\text{Driver Condition}) = 0.043 > IG(\text{Weather}) = 0.010$. Therefore, **Driver Condition** is selected as the root node.

ENTROPY CALCULATION — NODE SPLIT (2024 MIDSEM MAKEUP STYLE)

Q: A node has {12, 8} objects of class 1 and class 2. After splitting on attribute B: Left child {9,1}, Right child {3,7}. Compute entropy of parent, children, and IG.

1 Parent (20 total: 12 class1, 8 class2):

$$H = -\frac{12}{20} \log_2 \frac{12}{20} - \frac{8}{20} \log_2 \frac{8}{20} = -(0.6 \times (-0.737)) - (0.4 \times (-1.322))$$

$$= 0.442 + 0.529 = \mathbf{0.971 \text{ bits}}$$

2 Left child (10 total: 9 class1, 1 class2):

$$H_L = -\frac{9}{10} \log_2 \frac{9}{10} - \frac{1}{10} \log_2 \frac{1}{10} = 0.469$$

3 Right child (10 total: 3 class1, 7 class2):

$$H_R = -\frac{3}{10} \log_2 \frac{3}{10} - \frac{7}{10} \log_2 \frac{7}{10} = 0.881$$

4 Weighted entropy after split:

$$\frac{10}{20} (0.469) + \frac{10}{20} (0.881) = 0.235 + 0.441 = 0.675$$

✓ **IG = 0.971 – 0.675 = 0.296 bits**

The split on attribute B reduced impurity by 0.296 bits — a significant improvement.

26. Gain Ratio (C4.5)

💡 THE PROBLEM WITH INFORMATION GAIN

IG **favours attributes with many unique values** (e.g., Transaction ID, Name) — a unique-value attribute perfectly separates every example but creates useless leaves.

Gain Ratio normalises IG by the **Split Information (SplitInfo)** — the entropy of the attribute's value distribution:

$$\text{SplitInfo}(S, A) = - \sum_v \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}$$

$$\text{GainRatio}(S, A) = \frac{IG(S, A)}{\text{SplitInfo}(S, A)}$$

An attribute with many distinct values has high SplitInfo → low GainRatio → penalised.

🎯 IG VS GAIN RATIO — STANDARD EXAM ANSWER

Q: "What is the limitation of Information Gain, and how can it be addressed?"

→ IG tends to favour attributes with many distinct values (e.g., IDs, dates), which may not create meaningful splits. This is addressed by **Gain Ratio** (used in C4.5), which penalises attributes with high cardinality by dividing IG by the split information of the attribute itself.

27. Overfitting & Pruning in Decision Trees

💡 WHY DECISION TREES OVERFIT

A fully grown decision tree will achieve ~100% accuracy on training data by memorising every training example (creating very specific rules that only apply to those exact examples). On new data, it performs poorly.

Signs of overfitting in DT: training accuracy near 100% + test accuracy much lower.

The exam trap: "Assessing a decision tree using only training data is detrimental." → TRUE. The training accuracy can be 100% while test accuracy is much lower.

Technique	How it works	Advantage
Pre-pruning (Early stopping)	Stop growing the tree when: depth limit reached; node has too few samples; IG below threshold; chi-squared test fails	Fast, simple. But "myopic" — may stop before capturing XOR-like interactions
Post-pruning	Grow full tree → prune back branches that don't improve validation set performance (reduced error pruning)	Can capture complex interactions first, then simplify. Generally more effective

Handling Continuous Attributes (C4.5)

C4.5 handles continuous features by:

1. Sort instances by the continuous attribute value
2. Identify **candidate split points** = midpoints between adjacent values where the class label changes
3. Evaluate IG (or Gain Ratio) at each candidate threshold
4. Choose the threshold with the highest IG

Handling Missing Values in Decision Trees

C4.5 uses **probabilistic (fractional) splitting**: instances with missing attribute values are distributed fractionally across all branches, proportional to how other non-missing instances split.

[↑ Back to top](#)

Part VII — Exam Preparation

Sections 28–30 · Patterns, formula sheet & worked templates

Starting Guide — Part VII

What to do 24 hours before the exam:

1. Run through the formula sheet — tick off everything you know
2. Do one full entropy+IG calculation from scratch
3. Do one full confusion matrix calculation from scratch
4. Review the bias–variance table and regularization rules
5. Memorise Mitchell's definition and the T/P/E identification drill

28. Exam Patterns & Recurring Question Types

Analysis of **8 MidSem papers (2023–2026)** reveals these recurring questions:

Question Pattern	Papers appeared	Key points to cover
Mitchell's T/P/E identification + performance measures	Every paper	Name T, P, E; name 2 performance measures; identify learning type (supervised/unsupervised/RL)
Messy dataset — find 5–6 quality issues	Every paper (some use same dataset!)	Use the 6-type checklist: missing, outlier, inconsistent format, duplicate, unit mismatch, encoding inconsistency
Ridge regression high bias scenario	2023, 2023-2, 2024 Makeup, Sample QP	Equal high training & val error → underfitting → decrease λ
GD vs Normal Equation for large data	2023, 2023-2, Sample QP	Normal equation $O(n^3)$ → GD for large n
$J(\theta)=0$ true/false statements	2023, 2023-2, Sample QP	False: works on unseen; True: fits training exactly; False: $\theta \neq 0$ necessarily; False: J can be 0
Feature scaling T/F statements	2024 Makeup	Speeds convergence = True; guarantees global min = False; prevents singularity = False; prevents local optima = False
Confusion matrix → Precision, Recall, F1	Every paper	Apply formulas; interpret in context; discuss precision-recall trade-off
ID3 decision tree — entropy & IG calculation	Every paper	Compute $H(S)$, H per attribute value, weighted avg H , IG; choose root with highest IG

Question Pattern	Papers appeared	Key points to cover
Overfit DT — why training accuracy misleads	Sample QP, 2023, 2024	Full tree achieves ~100% training accuracy but generalises poorly → always evaluate on test/validation data
Lasso vs Ridge — which for feature selection?	2024 Regular, 2024 Makeup	Lasso for sparse/irrelevant features; Ridge when all features matter
Entropy bits question (4 classes uniform)	2024 Regular	$H = \log_2(4) = 2$ bits; non-uniform → lower entropy; rare event needs more bits
Overfitting symptoms & sigmoid overfitting	2024 Regular, Makeup	High train acc + low val acc; large weights → step function → overfit
Polynomial regression Model A/B/C/D comparison	2024 Regular, 2024 Makeup	Linear = underfitting; degree-8 no reg = overfitting; degree-8 + Lasso = feature selection; rank RMSE train vs test
Stratified split for imbalanced dataset	2024 Regular	Random split risks distorting class distribution; stratified split preserves class ratios in each fold
Why LR fails for classification	2024 Makeup, 2025	Outputs unbounded (not a probability); MSE on binary → non-convex cost; sensitive to outliers. Fix: sigmoid + log-loss

29. Quick-Reference Formula Sheet

Core ML Formulas

MITCHELL'S DEFINITION

T = Task; P = Performance; E = Experience.
Performance on T improves with E, measured by P.

MSE COST FUNCTION

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

GD UPDATE RULE

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial J}{\partial \theta_j}$$

$$\text{For MSE: gradient} = \frac{1}{m} \sum (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}$$

NORMAL EQUATION

$$\theta = (X^T X)^{-1} X^T y$$

Use only for small n. Cost: $O(n^3)$

RIDGE (L2)

LASSO (L1)

$$J = \text{MSE} + \lambda \sum_j \theta_j^2$$

Shrinks all coeff. Never zero.

$$J = \text{MSE} + \lambda \sum_j |\theta_j|$$

Can zero out coefficients. Feature selection.

SIGMOID

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Maps $\mathbb{R} \rightarrow (0,1)$. Decision boundary at $z=0$.

LOG-LOSS

$$J = -\frac{1}{m} \sum [y \log \hat{p} + (1 - y) \log(1 - \hat{p})]$$

LOG-ODDS

$$\log \frac{P(y=1)}{P(y=0)} = \theta^T \mathbf{x}$$

θ_j = change in log-odds per unit x_j increase

R-SQUARED

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

Range: $(-\infty, 1]$. 1=perfect, 0=mean baseline.

Z-SCORE SCALING

$$x' = \frac{x - \mu}{\sigma}$$

Output: mean=0, std=1

IQR OUTLIER RULE

Outlier if: $x < Q1 - 1.5 \times \text{IQR}$ or $x > Q3 + 1.5 \times \text{IQR}$

$\text{IQR} = Q3 - Q1$

POLYNOMIAL REGRESSION (DEGREE D)

$$\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2 + \dots + \theta_d x^d$$

Still linear in θ ; features engineered as powers of x .

MIN-MAX NORMALISATION

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Maps all values to $[0, 1]$.

Evaluation Metric Formulas

ACCURACY

$$\frac{TP+TN}{TP+FP+FN+TN}$$

⚠ Misleading on imbalanced data

PRECISION

$$\frac{TP}{TP+FP}$$

Minimise if FP is costly (spam filter)

RECALL (TPR)

$$\frac{TP}{TP+FN}$$

Maximise if FN is costly (cancer, fraud)

SPECIFICITY (TNR)

$$\frac{TN}{TN+FP}$$

FPR (X-AXIS OF ROC)

$$\frac{FP}{FP+TN} = 1 - \text{Specificity}$$

F1-SCORE

$$\frac{2 \cdot P \cdot R}{P+R}$$

Best summary for imbalanced data

Decision Tree Formulas

ENTROPY

$$H(S) = - \sum_c p_c \log_2 p_c$$

H=0 (pure), H=1 (50/50 binary)

INFORMATION GAIN

$$IG(S, A) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v)$$

Higher IG → better split attribute

SPLIT INFORMATION

$$\text{SplitInfo} = - \sum_v \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}$$

GAIN RATIO (C4.5)

$$\text{GainRatio} = \frac{IG(S, A)}{\text{SplitInfo}(S, A)}$$

Penalises high-cardinality attributes

ENTROPY (2 CLASSES)

If p = fraction of class 1:

$$H = -p \log_2 p - (1 - p) \log_2 (1 - p)$$

BITS FOR K CLASSES

Minimum bits to encode k equally likely classes =

$$\log_2 k$$

4 classes → $\log_2 4 = 2$ bits

30. Worked Templates — One Per Question Type

Template A — Mitchell's T/P/E (Guaranteed Q1)

TEMPLATE: WEATHER PREDICTION

Q: A system uses historical weather data to predict whether tomorrow will be Sunny, Rainy, or Windy. What is T, P, E? Name 2 performance measures.

- T Task:** Predict tomorrow's weather as Sunny, Rainy, or Windy. This is a *supervised classification* task.
- P Performance:** The fraction of days on which the weather is predicted correctly.
Performance measures: *Accuracy, Precision/Recall, F1-score, AUC-ROC*.
- E Experience:** The historical weather dataset — past records of weather conditions and their actual outcomes.
-  Learning type: **Supervised learning** (labelled historical data). If asked about rainfall amount (continuous) → Regression, P = MSE or R².

Template B — Data Quality (Guaranteed Q2)

5-SECOND CHECKLIST FOR ANY DATASET TABLE

1. Scan for **blank/empty cells** → missing value
2. Scan for **extreme numbers** (age 350, CGPA 87.5) → outlier
3. Scan for **same column, different formats** (dates, blood groups) → inconsistent format
4. Scan for **identical rows** → duplicate
5. Scan for **wrong units or wrong encoding** (lbs vs kg, 1/0 vs Yes/No) → unit mismatch / encoding error
6. Scan for **non-sensical entries** (SysUsr789 as a name, negative weight) → invalid data / noise

Template C — Entropy + IG Calculation (Guaranteed Q6)

STEP-BY-STEP ENTROPY TEMPLATE

Given: Target class has [Yes=9, No=5] → 14 total. Feature A has {val1: 6 total (5Y, 1N), val2: 8 total (4Y, 4N)}.

1 Parent entropy $H(S)$:

$$p(\text{Yes}) = 9/14, p(\text{No}) = 5/14$$

$$H(S) = -\frac{9}{14}\log_2 \frac{9}{14} - \frac{5}{14}\log_2 \frac{5}{14}$$

$$\approx -(0.643 \times (-0.638)) - (0.357 \times (-1.485)) = 0.410 + 0.530 = \mathbf{0.940 \text{ bits}}$$

2 Entropy of val1 (6 total: 5Y, 1N):

$$H_{v1} = -\frac{5}{6}\log_2 \frac{5}{6} - \frac{1}{6}\log_2 \frac{1}{6} = 0.650$$

3 Entropy of val2 (8 total: 4Y, 4N):

$$H_{v2} = -\frac{4}{8}\log_2 \frac{4}{8} - \frac{4}{8}\log_2 \frac{4}{8} = 1.000 \text{ (maximum — 50-50 split)}$$

4 Weighted average entropy:

$$\text{Weighted} = \frac{6}{14}(0.650) + \frac{8}{14}(1.000) = 0.279 + 0.571 = 0.850$$

✓ **$IG(A) = H(S) - \text{Weighted} = 0.940 - 0.850 = 0.090 \text{ bits}$**

Compare with IG of other attributes; the attribute with the **highest IG** becomes the root node.

Template D — Confusion Matrix (Guaranteed Q5)

MEDICAL DIAGNOSIS — RARE DISEASE

Given: Model A: TP=180, FP=200, FN=20, TN=9600 (out of 10,000 patients). Calculate Precision, Recall, F1, Specificity. Which model is better for medical use?

- 1 **Precision:** $\frac{180}{180+200} = \frac{180}{380} = 47.4\%$ — Less than half of "disease" predictions are correct.
 - 2 **Recall:** $\frac{180}{180+20} = \frac{180}{200} = 90.0\%$ — Catches 90% of actual disease cases. Very important in medicine.
 - 3 **Specificity:** $\frac{9600}{9600+200} = 97.9\%$
 - 4 **F1:** $\frac{2 \times 0.474 \times 0.900}{0.474 + 0.900} = \frac{0.853}{1.374} = 62.1\%$
- ✅ **Medical context:** In disease detection, **FN (missed disease) is far more costly** than FP (unnecessary follow-up test). A model with higher Recall is preferred even at the cost of lower Precision. If comparing models, choose the one with highest Recall/F1 for the disease class.

Template E — Bias–Variance Diagnosis

Observation in exam question	Diagnosis	Action
Train error $\approx$ val error, both HIGH	High bias (underfitting)	Decrease λ , add features, more complex model, more training time
Train error LOW, val error HIGH (large gap)	High variance (overfitting)	Increase λ , get more data, simpler model, early stopping, dropout
Training error continuously decreases, test error increases after iteration 5	Overfitting in progress	Apply early stopping at iteration 5
Large coefficients ($\beta_1=85$, $\beta_2=-72$) + train acc 0.94, val acc 0.70	Overfitting (large weights $\rightarrow$ steep sigmoid)	L1/L2 regularization, early stopping, more data
$R^2(\text{test}) = 0.42$ for polynomial degree-8 model	Overfitting (worse than simpler model on test)	Add regularization, reduce polynomial degree

1. Mitchell's T/P/E — I can identify these for any scenario ✓
2. The 6 data quality issue types — I can spot them in a table ✓
3. Entropy formula — I can compute $H(S)$ from scratch ✓
4. IG formula — I know it's $H(\text{parent}) - \text{weighted } H(\text{children})$ ✓
5. Confusion matrix — TP/FP/FN/TN, Precision = $TP/(TP+FP)$, Recall = $TP/(TP+FN)$ ✓
6. Ridge vs Lasso — Lasso zeros coefficients (feature selection) ✓
7. Bias-variance: equal high errors → underfitting → decrease λ ✓
8. $J(\theta)=0$ → overfitting, not a good model ✓
9. GD vs Normal Equation: large dataset → GD (normal eq is $O(n^3)$) ✓
10. Scaling: speeds GD convergence = TRUE; guarantees global min = FALSE ✓

[↑ Back to top](#)